ЛР5. Градиентная оптимизация линейной регрессии — пошаговый гайд

Своя линейная регрессия, ≥ 3 функционала ошибки (MSE обязательно), ≥ 3 градиентных метода, критерий $\|\nabla Q\|_2 < \varepsilon$, подбор шага, обусловленность гессиана, графики динамики, сравнение со scikit-learn. Всё — на двух версиях данных: без масштабирования и с масштабированием.

Что сдавать

Раздел	Содержание
2.1	Предобработка; две версии признаков: без масштабирования и с масштабированием
2.2	Архитектура: абстрактные классы функционала и оптимизатора, модель независима от них
2.3	Свой класс линейной регрессии (`fit`, `predict`), свободный член
2.4	≥ 3 функционала: эмпирический функционал, градиент, гессиан, точки недифференцируемости — **в TeX, в отдельных Markdown-ячейках**
2.5	≥ 3 оптимизатора с общим интерфейсом; сохранение истории
2.6	Эксперимент на немасштабированных данных: критерий, подбор η (и C), $\kappa(H)$, три графика, RMSE на тесте
2.7	Сравнение с моделями sklearn
2.8	Повтор для масштабированных данных и выводы о влиянии масштабирования

Создание ноутбука — раздел 0 гайда ЛР3. Лекционный ноутбук `Лабы/ЛР5/GD.py` (marimo) содержит реализации GD, SGD, Momentum, AdaGrad, RMSProp и Adam — полезно открыть рядом.

2.1. Подготовка данных

Нужно ≥ 10 000 объектов. Melbourne Housing (13 580) подходит; берём `melb_clean.csv` из ЛР3. Цель — $\ln(\text{Price})$: цена логнормальна, а в логарифме ошибки соизмеримы, и значения функционала не астрономические.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('melb_clean.csv', parse_dates=['Date'])
df = pd.get_dummies(df, columns=['Type', 'Regionname'], drop_first=True, dtype=float)
# Age (= год продажи − YearBuilt) сюда НЕ берём: он почти линейно зависим от YearBuilt
# и делает гессиан плохо обусловленным даже после масштабирования. Он понадобится в ЛР6.
feature_cols = (['Rooms', 'Distance', 'Bathroom', 'Car', 'Landsize', 'BuildingArea', 'YearBuilt',
                 'Lattitude', 'Longtitude', 'Propertycount']
                + [c for c in df.columns if c.startswith(('Type_', 'Regionname_'))])
```

```
X_all = df[feature_cols].to_numpy(float)
y_all = np.log(df['Price'].to_numpy(float))

# train / validation / test = 60 / 20 / 20
X_tr, X_tmp, y_tr, y_tmp = train_test_split(X_all, y_all, test_size=0.4, random_state=42)
X_val, X_te, y_val, y_te = train_test_split(X_tmp, y_tmp, test_size=0.5, random_state=42)

scaler = StandardScaler().fit(X_tr)                    # статистики — только по train
data = {
    'raw':    (X_tr, X_val, X_te),
    'scaled': (scaler.transform(X_tr), scaler.transform(X_val), scaler.transform(X_te)),
}
print(X_tr.shape, X_val.shape, X_te.shape)
```

2.2–2.3. Архитектура и модель

Три независимых компонента: **функционал** знает только, как считать Q, ∇Q и H по матрице X и вектору y; **оптимизатор** знает только, как сделать шаг по градиенту; **модель** соединяет их и хранит историю.

```
class Loss(ABC):
    """Эмпирический функционал Q(w) = (1/ℓ) Σ L(y_i, <w, x_i>)."""
    name = 'loss'

    @abstractmethod
    def value(self, X, y, w): ...

    @abstractmethod
    def grad(self, X, y, w): ...

    @abstractmethod
    def hessian(self, X, y, w):
        """Матрица вторых производных или None, если она не существует/равна 0 почти всюду."""


class Optimizer(ABC):
    name = 'opt'

    def reset(self, dim):
        self.k = 0

    @abstractmethod
    def step(self, w, g):
        """Вернуть новый вектор весов по текущим весам w и градиенту g."""


class LinearRegressionGD:
```

```
    """Линейная регрессия; свободный член — единичный признак (первый столбец)."""

    def __init__(self, loss, optimizer, eps=1e-6, max_iter=20000, batch_size=None,
                 track_every=50, random_state=0):
        self.loss, self.opt, self.eps, self.max_iter = loss, optimizer, eps, max_iter
        self.batch_size, self.track_every = batch_size, track_every
        self.rng = np.random.default_rng(random_state)

    @staticmethod
    def _add_bias(X):
        return np.hstack([np.ones((X.shape[0], 1)), X])

    def fit(self, X, y):
        Xb = self._add_bias(X)
        n, d = Xb.shape
        w = np.zeros(d)
        self.opt.reset(d)
        self.history = {'loss': [], 'grad_norm': [], 'kappa_iter': [], 'kappa': []}
        self.converged, self.diverged = False, False
        for k in range(self.max_iter):
            full_g = self.loss.grad(Xb, y, w)              # полный градиент — для критерия
            gn = np.linalg.norm(full_g)
            self.history['loss'].append(self.loss.value(Xb, y, w))
            self.history['grad_norm'].append(gn)
            if k % self.track_every == 0:
                self.history['kappa_iter'].append(k)
                self.history['kappa'].append(condition_number(self.loss.hessian(Xb, y, w)))
            if not np.isfinite(gn):
                self.diverged = True
                break
            if gn < self.eps:
                self.converged = True
                break
            if self.batch_size:                            # mini-batch градиент
                idx = self.rng.choice(n, self.batch_size, replace=False)
                g = self.loss.grad(Xb[idx], y[idx], w)
            else:
                g = full_g
            w = self.opt.step(w, g)
        self.w_, self.n_iter_ = w, k + 1
        self.intercept_, self.coef_ = w[0], w[1:]
        return self

    def predict(self, X):
        return self._add_bias(X) @ self.w_
```

```
def condition_number(H, tol=1e-15):
    """κ(H) = λmax/λmin для симметричной неотрицательно определённой H; inf, если вырождена."""
    if H is None:
        return np.nan                       # гессиан не существует / нулевой почти всюду
    lam = np.linalg.eigvalsh(H)
    if not np.all(np.isfinite(lam)) or lam.max() <= 0:
        return np.inf
    return np.inf if lam.min() <= tol * lam.max() else lam.max() / lam.min()


def rmse(y, p):
    return float(np.sqrt(np.mean((y - p) ** 2)))
```

2.4. Функционалы ошибки

Обозначения: $r = Xw - y \in \mathbb{R}^\ell$ — вектор отклонений, ℓ — число объектов. Каждый вывод оформите **отдельной Markdown-ячейкой** с TeX — ниже готовые формулы (подробные выводы — в теории ЛР5).

MSE.

$$Q(w) = \frac{1}{\ell}\|Xw - y\|_2^2, \quad \nabla Q = \frac{2}{\ell}X^T(Xw - y), \quad H = \frac{2}{\ell}X^T X.$$

Дифференцируем всюду; H не зависит от w.

MAE.

$$Q(w) = \frac{1}{\ell}\|Xw - y\|_1, \quad \nabla Q = \frac{1}{\ell}X^T \operatorname{sign}(Xw - y), \quad H = 0 \text{ почти всюду}.$$

Не дифференцируем на гиперплоскостях $x_i^T w = y_i$ — там используется субградиент.

Huber ($\delta > 0$):

$$Q(w) = \frac{1}{\ell}\sum_i L_\delta(r_i), \quad \nabla Q = \frac{1}{\ell}X^T\psi(r), \;\; \psi(r_i) = \begin{cases} r_i, & |r_i| \le \delta \\ \delta \operatorname{sign} r_i, & |r_i| > \delta \end{cases}, \quad H = \frac{1}{\ell}X^T D X,$$

где $D = \operatorname{diag}\big([\,|r_i| \le \delta\,]\big)$. Гессиан не существует при $|r_i| = \delta$.

Log-cosh.

$$Q(w) = \frac{1}{\ell}\sum_i \ln\cosh r_i, \quad \nabla Q = \frac{1}{\ell}X^T \tanh(r), \quad H = \frac{1}{\ell}X^T \operatorname{diag}\big(1 - \tanh^2 r_i\big) X.$$

Дважды дифференцируем всюду; $H \succ 0$ при полном ранге X.

```
class MSELoss(Loss):
    name = 'MSE'
    def value(self, X, y, w):   r = X @ w - y; return float(r @ r / len(y))
    def grad(self, X, y, w):    return 2 / len(y) * X.T @ (X @ w - y)
    def hessian(self, X, y, w): return 2 / len(y) * X.T @ X


class MAELoss(Loss):
    name = 'MAE'
```

```
    def value(self, X, y, w):   return float(np.mean(np.abs(X @ w - y)))
    def grad(self, X, y, w):    return X.T @ np.sign(X @ w - y) / len(y)
    def hessian(self, X, y, w): return None


class HuberLoss(Loss):
    name = 'Huber'
    def __init__(self, delta=0.3): self.delta = delta
    def value(self, X, y, w):
        a = np.abs(X @ w - y); d = self.delta
        return float(np.mean(np.where(a <= d, 0.5 * a ** 2, d * (a - 0.5 * d))))
    def grad(self, X, y, w):
        r = X @ w - y
        return X.T @ np.clip(r, -self.delta, self.delta) / len(y)
    def hessian(self, X, y, w):
        m = (np.abs(X @ w - y) <= self.delta).astype(float)
        return (X * m[:, None]).T @ X / len(y)


class LogCoshLoss(Loss):
    name = 'Log-cosh'
    def value(self, X, y, w):
        r = X @ w - y
        return float(np.mean(np.logaddexp(r, -r) - np.log(2)))   # устойчивый ln cosh
    def grad(self, X, y, w):    return X.T @ np.tanh(X @ w - y) / len(y)
    def hessian(self, X, y, w):
        s = 1 - np.tanh(X @ w - y) ** 2
        return (X * s[:, None]).T @ X / len(y)
```

`np.clip(r, -δ, δ)` — это и есть $\psi(r)$; `logaddexp(r, -r) - ln 2` $= \ln(e^r + e^{-r}) - \ln 2 = \ln\cosh r$ без переполнения.

2.5. Оптимизаторы

```
class GD(Optimizer):
    """Постоянный шаг или затухающий η_k = C / (k + k0)."""
    def __init__(self, lr=1e-2, C=None, k0=100):
        self.lr, self.C, self.k0 = lr, C, k0
        self.name = 'GD' if C is None else 'GD (затух.)'
    def eta(self):
        return self.lr if self.C is None else self.C / (self.k + self.k0)
    def step(self, w, g):
        w = w - self.eta() * g; self.k += 1; return w


class Momentum(Optimizer):
    name = 'Momentum'
```

```
    def __init__(self, lr=1e-2, beta=0.9): self.lr, self.beta = lr, beta
    def reset(self, dim): super().reset(dim); self.v = np.zeros(dim)
    def step(self, w, g):
        self.v = self.beta * self.v + g; self.k += 1
        return w - self.lr * self.v


class AdaGrad(Optimizer):
    name = 'AdaGrad'
    def __init__(self, lr=1e-1, eps=1e-8): self.lr, self.eps = lr, eps
    def reset(self, dim): super().reset(dim); self.G = np.zeros(dim)
    def step(self, w, g):
        self.G += g ** 2; self.k += 1
        return w - self.lr * g / (np.sqrt(self.G) + self.eps)


class RMSProp(Optimizer):
    name = 'RMSProp'
    def __init__(self, lr=1e-2, beta=0.9, eps=1e-8): self.lr, self.beta, self.eps = lr, beta, eps
    def reset(self, dim): super().reset(dim); self.v = np.zeros(dim)
    def step(self, w, g):
        self.v = self.beta * self.v + (1 - self.beta) * g ** 2; self.k += 1
        return w - self.lr * g / (np.sqrt(self.v) + self.eps)


class Adam(Optimizer):
    name = 'Adam'
    def __init__(self, lr=1e-2, beta1=0.9, beta2=0.999, eps=1e-8):
        self.lr, self.b1, self.b2, self.eps = lr, beta1, beta2, eps
    def reset(self, dim): super().reset(dim); self.m = np.zeros(dim); self.v = np.zeros(dim)
    def step(self, w, g):
        self.k += 1
        self.m = self.b1 * self.m + (1 - self.b1) * g
        self.v = self.b2 * self.v + (1 - self.b2) * g ** 2
        m_hat = self.m / (1 - self.b1 ** self.k)
        v_hat = self.v / (1 - self.b2 ** self.k)
        return w - self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
```

Для **mini-batch SGD** отдельный класс не нужен: это `GD` + `LinearRegressionGD(batch_size=256)`. Критерий остановки по-прежнему считается по **полному** градиенту.

2.6. Эксперимент на немасштабированных данных

2.6.1. Критерий сходимости

$\|\nabla Q(w^{(k)})\|_2 < \varepsilon$, $\varepsilon = 10^{-6}$, одинаковый для всех запусков. Плюс страховка `max_iter`: без масштабирования GD за разумное время **не сойдётся** — это и есть результат эксперимента. Для MAE

полный градиент вблизи минимума не стремится к нулю (это субградиент, его норма «прыгает»), поэтому критерий может не сработать ни при каком шаге — отметьте это в выводах.

2.6.2. Подбор шага по валидации

```
EPS = 1e-6
losses = {'MSE': MSELoss(), 'MAE': MAELoss(), 'Huber': HuberLoss(0.3), 'Log-cosh': LogCoshLoss()}
opt_factories = {
    'GD':       lambda lr: GD(lr=lr),
    'Momentum': lambda lr: Momentum(lr=lr, beta=0.9),
    'Adam':     lambda lr: Adam(lr=lr),
}
# Сетка методички 1e-5…1 ориентировочная. Без масштабирования λmax(H) ~ 1e8
# (Propertycount ~ 1e4 в квадрате), допустимый шаг η < 2/λmax ≈ 1e-8 — сетку приходится
# расширять вниз. Сам этот факт — важный результат эксперимента.
LR_GRID = {'raw':    [10.0 ** p for p in range(-12, -1)],
           'scaled': [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1]}

def tune(version, loss, make_opt, grid=None, max_iter=3000):
    Xtr, Xv, _ = data[version]
    grid = LR_GRID[version] if grid is None else grid
    best = None
    for lr in grid:
        with np.errstate(all='ignore'):
            m = LinearRegressionGD(loss, make_opt(lr), eps=EPS, max_iter=max_iter,
                                   track_every=10 ** 9).fit(Xtr, y_tr)
            score = rmse(y_val, m.predict(Xv)) if not m.diverged else np.inf
        if not np.isfinite(score):
            score = np.inf
        if best is None or score < best[1]:
            best = (lr, score)
    return best

results = []
for ln, loss in losses.items():
    for on, mk in opt_factories.items():
        lr, val_rmse = tune('raw', loss, mk)
        results.append({'данные': 'raw', 'функционал': ln, 'метод': on, 'lr': lr, 'RMSE val':
val_rmse})
tuned_raw = pd.DataFrame(results)
tuned_raw
```

Для затухающего шага $\eta_k = C/(k + k_0)$, $k_0 = 100$, перебираем C. На масштабированных данных — $C \in \{10^{-3}, \dots, 10^{2}\}$, как в методичке. Без масштабирования сетку сдвигаем так же, как для η: первый шаг C/k_0 должен быть меньше $2/\lambda_{\max}$.

```
C_GRID = {v: [lr * 100 for lr in LR_GRID[v]] for v in LR_GRID}
best_C = {ln: tune('raw', loss, lambda C: GD(C=C, k0=100), grid=C_GRID['raw']) for ln, loss in
losses.items()}
best_C
```

Сетка подбора на `max_iter=3000` — компромисс по времени. Итоговые модели обучите с большим `max_iter`.

2.6.3–2.6.4. Итоговое обучение, обусловленность, графики

```
def train_final(version, tuned, max_iter=20000):
    Xtr, _, Xte = data[version]
    models = {}
    for _, row in tuned.iterrows():
        loss = losses[row['функционал']]
        opt = opt_factories[row['метод']](row['lr'])
        with np.errstate(all='ignore'):
            models[(row['функционал'], row['метод'])] = LinearRegressionGD(
                loss, opt, eps=EPS, max_iter=max_iter, track_every=100).fit(Xtr, y_tr)
    return models

models_raw = train_final('raw', tuned_raw)

def plot_dynamics(models, title):
    for ln in losses:
        fig, axes = plt.subplots(1, 3, figsize=(16, 3.6))
        for (l, on), m in models.items():
            if l != ln:
                continue
            h = m.history
            axes[0].semilogy(h['grad_norm'], label=on)
            axes[1].semilogy(h['kappa_iter'], h['kappa'], marker='.', label=on)
            axes[2].semilogy(np.array(h['loss']) - min(h['loss']) + 1e-12, label=on)
        axes[0].axhline(EPS, color='k', ls='--', label='ε')
        axes[0].set_title(f'{ln}: ‖∇Q(w_k)‖₂'); axes[1].set_title('κ(H(w_k))')
        axes[2].set_title('Q(w_k) − min Q')
        for ax in axes:
            ax.set_xlabel('k'); ax.legend(fontsize=8)
        fig.suptitle(title); plt.tight_layout(); plt.show()

plot_dynamics(models_raw, 'Без масштабирования')
```

Для MAE гессиан почти всюду нулевой, поэтому κ не определена (`nan`, на графике не рисуется). Для Huber $\kappa = \infty$, если в «квадратичную» зону $|r_i| \le \delta$ попало слишком мало объектов. Оба случая укажите в выводах.

Что анализировать:

- **скорость:** сколько итераций до ε (или не сошлось);

- **колебания:** зигзаги нормы градиента у GD с большим шагом, «пилообразность» Momentum;
- **функционал:** MSE и Log-cosh гладкие, MAE «ступенчатый»;
- **обусловленность:** без масштабирования $\kappa(H)$ огромна (10^8 и больше — `YearBuilt` ≈ 2000 рядом с единичным столбцом свободного члена), поэтому GD обязан брать крошечный шаг $\eta < 2/\lambda_{\max}$, и спуск вдоль направления $\lambda_{\min}$ почти стоит.

```
pd.DataFrame([{'функционал': l, 'метод': o, 'итераций': m.n_iter_, 'сошёлся': m.converged,
               'κ в конце': m.history['kappa'][-1]} for (l, o), m in models_raw.items()])
```

2.6.5. Качество на тесте

```
def test_table(models, version):
    Xte = data[version][2]
    return pd.DataFrame([{'данные': version, 'функционал': l, 'метод': o,
                          'RMSE test (ln Price)': rmse(y_te, m.predict(Xte))}
                         for (l, o), m in models.items()]) \
             .pivot(index='функционал', columns='метод', values='RMSE test (ln Price)').round(4)

test_table(models_raw, 'raw')
```

RMSE в шкале $\ln$: значение 0,25 соответствует типичной ошибке около $e^{0{,}25} - 1 \approx 28\,\%$ цены.

2.7. Сравнение со scikit-learn

Наш функционал	Модель sklearn	Комментарий
MSE	`LinearRegression()`	аналитическое решение — эталон
MSE	`SGDRegressor(loss='squared_error', penalty=None)`	стохастический спуск, подбираем `eta0`
MAE	`QuantileRegressor(quantile=0.5, alpha=0)`	медианная регрессия = MAE
MAE	`SGDRegressor(loss='epsilon_insensitive', epsilon=0, penalty=None)`	при $\epsilon = 0$ это ровно $\lvert r \rvert$
Huber	`SGDRegressor(loss='huber', epsilon=δ, penalty=None)`	тот же δ
Huber	`HuberRegressor(epsilon=..., alpha=0)`	другая параметризация (оценивает масштаб σ)
Log-cosh	—	в sklearn нет

```
from sklearn.linear_model import LinearRegression, SGDRegressor, QuantileRegressor, HuberRegressor

def sk_eval(version):
    Xtr, Xv, Xte = data[version]
    rows = []
    lr_model = LinearRegression().fit(Xtr, y_tr)
```

```
    rows.append(('MSE', 'LinearRegression', '—', rmse(y_te, lr_model.predict(Xte))))
    for ln, kw in [('MSE', dict(loss='squared_error')),
                   ('MAE', dict(loss='epsilon_insensitive', epsilon=0.0)),
                   ('Huber', dict(loss='huber', epsilon=0.3))]:
        best = None
        for eta0 in [1e-5, 1e-4, 1e-3, 1e-2]:
            with np.errstate(all='ignore'):
                m = SGDRegressor(penalty=None, eta0=eta0, learning_rate='invscaling',
                                 max_iter=2000, tol=1e-6, random_state=0, **kw).fit(Xtr, y_tr)
                s = rmse(y_val, m.predict(Xv))
            if np.isfinite(s) and (best is None or s < best[1]):
                best = (m, s, eta0)
        if best:
            rows.append((ln, 'SGDRegressor', best[2], rmse(y_te, best[0].predict(Xte))))
    sub = np.random.default_rng(0).choice(len(Xtr), min(3000, len(Xtr)), replace=False)
    q = QuantileRegressor(quantile=0.5, alpha=0, solver='highs').fit(Xtr[sub], y_tr[sub])
    rows.append(('MAE', 'QuantileRegressor (3000 строк)', '—', rmse(y_te, q.predict(Xte))))
    hub = HuberRegressor(epsilon=1.35, alpha=0, max_iter=2000).fit(Xtr, y_tr)
    rows.append(('Huber', 'HuberRegressor', '—', rmse(y_te, hub.predict(Xte))))
    return pd.DataFrame(rows, columns=['функционал', 'модель sklearn', 'eta0', 'RMSE test'])

sk_eval('raw')
```

`QuantileRegressor` решает задачу линейного программирования и на 8 000 строк работает долго, поэтому обучаем его на подвыборке. Без масштабирования `SGDRegressor` часто расходится — это нормально, отметьте в выводах.

2.8. Повтор с масштабированием

```
results = []
for ln, loss in losses.items():
    for on, mk in opt_factories.items():
        lr, val_rmse = tune('scaled', loss, mk)
        results.append({'данные': 'scaled', 'функционал': ln, 'метод': on, 'lr': lr, 'RMSE val':
val_rmse})
tuned_scaled = pd.DataFrame(results)
models_scaled = train_final('scaled', tuned_scaled)
plot_dynamics(models_scaled, 'Со стандартизацией')
display(test_table(models_scaled, 'scaled'))
sk_eval('scaled')
```

```
pd.DataFrame([{'функционал': l, 'метод': o,
               'итераций raw': models_raw[(l, o)].n_iter_, 'итераций scaled': models_scaled[(l,
o)].n_iter_,
               'к raw': models_raw[(l, o)].history['kappa'][-1],
               'к scaled': models_scaled[(l, o)].history['kappa'][-1]}
```

```
              for (l, o) in models_raw])
```

Ожидаемые выводы:

- после стандартизации $\kappa(H)$ падает на много порядков, допустимый шаг растёт, GD сходится за сотни–тысячи итераций вместо «не сошёлся»;
- адаптивные методы (Adam, AdaGrad, RMSProp) меньше страдают от плохого масштаба, потому что нормируют шаг по каждой координате — но не от корреляции признаков. Без масштабирования только Adam даёт приличное качество;
- Adam с постоянным шагом вблизи минимума «дрожит»: нормированный шаг $\eta\,\hat{m}/\sqrt{\hat{v}}$ не стремится к нулю, поэтому критерий $\|\nabla Q\| < 10^{-6}$ может не выполниться. Лечится уменьшением η или затуханием шага;
- качество на тесте у сошедшихся моделей с разными функционалами близко; MAE и Huber устойчивее к выбросам цены;
- результаты собственной MSE-модели совпадают с `LinearRegression` (проверка корректности реализации).

Чек-лист

- [ ] Две версии данных; масштабирование обучено только на train.
- [ ] Абстрактные `Loss` и `Optimizer`; модель не зависит от конкретного функционала.
- [ ] ≥ 3 функционала (MSE + один с ненулевой второй производной, например Huber или Log-cosh) — вывод Q, ∇Q, H, точки недифференцируемости в TeX.
- [ ] ≥ 3 оптимизатора, история (loss, $\|\nabla Q\|$, κ).
- [ ] Одинаковый ε, подбор η / C по валидации.
- [ ] Три графика на каждую пару «функционал × метод», линия $y = \varepsilon$; отмечены случаи без гессиана / вырожденного гессиана.
- [ ] RMSE на тесте для всех моделей.
- [ ] Сравнение со sklearn с подбором гиперпараметров.
- [ ] Повтор на масштабированных данных и сравнение.